

 RECORDATORIO: MOSTRAR DNI A CÁMARA

LudiGest

Plataforma Intermodular de Gestión de Ludotecas

Alumno: Alejandro Muñoz de la Sierra

Profesor: Alejandro Rioja Chocron

Desarrollo de Aplicaciones Multiplataforma | 11 de Mayo de 2026

1. Problema y 2. Objetivos

El Problema Detectado

Las asociaciones de juegos sufren un caos logístico: inventarios en Excel desactualizados, metadatos tecleados a mano propensos a errores y falta de trazabilidad en los préstamos físicos.

Objetivos de LudiGest

-  **Automatización:** Ingesta de datos directa desde la API de BoardGameGeek (BGG).
-  **Trazabilidad Física:** Separar la "ficha del juego" de la "caja física" (Ejemplar).
-  **Reactividad:** Búsqueda y filtrado instantáneo para socios sin saturar la base de datos.



[INSERTA AQUÍ LA CAPTURA DEL CATÁLOGO]
(Para mostrar cómo se ve el objetivo final cumplido)

3. Tecnologías y su Justificación

Decisiones técnicas basadas en el rendimiento, la escalabilidad y la separación de responsabilidades.



Backend: Spring Boot 4

¿Por qué? Porque permite crear una API RESTful robusta y segura. Usa Jakarta Validation para blindar los datos antes de guardarlos y aísla la lógica de negocio.



Frontend: JavaFX 21

¿Por qué? Ofrece renderizado por hardware y nos permite separar la vista (.fxml) de la lógica (Controllers), inyectando CSS para un tema "Océano y Coral" unificado.



BD: MySQL 8 & Hibernate

¿Por qué? MySQL asegura integridad relacional (ACID). Hibernate mapea nuestros objetos Java a tablas SQL automáticamente, ahorrando código repetitivo (Boilerplate).

4. Arquitectura del Sistema

Cliente – Servidor (N-Tier)

LudiGest está completamente desacoplado. El frontend podría ser una App Móvil mañana mismo porque el backend es independiente.

 **Capa de Presentación:** JavaFX realiza peticiones HTTP.

 **Capa de Control (REST):** Los Controllers validan y enrutan.

 **Capa de Servicio:** Aquí reside el BggSyncService que piensa y calcula.

 **Capa de Persistencia:** Los Repositories hablan con MySQL.



[INSERTA AQUÍ LA CAPTURA DE LAS CARPETAS DEL PROYECTO]
(Las capturas de tu Eclipse mostrando model, service, controller, etc.)

5. Estructura de la Base de Datos

Diseño relacional estructurado para evitar redundancia y garantizar la trazabilidad de inventario.

juegos_referencia (La Ficha Teórica)

PK id (bigint)	Clave Primaria
id_bgg (bigint)	UNIQUE (BoardGameGeek)
titulo, dureza, max_jugadores...	Metadatos

ejemplares (La Caja Física)

PK id (bigint)	Clave Primaria
codigo_local (varchar)	UNIQUE (Pegatina)
FK juego_referencia_id	-> juegos_referencia(id)
estado, balda, en_venta	Control físico

usuarios (Socios y Admins)

PK id (bigint)	Clave Primaria
username, password, rol	Credenciales

prestamos (Transaccional)

PK id (bigint)	Clave Primaria
FK ejemplar_id	-> ejemplares(id)
FK usuario_id	-> usuarios(id)
fecha_prestamo, fecha_limite	Trazabilidad temporal

7. Dificultades y Soluciones (I) – Backend

⚠ Reto: Ingesta BGG y Data Truncation

Al descargar juegos externos, el XML contenía descripciones enormes y símbolos extraños que colapsaban MySQL (*Data Truncation Exception*).

Solución Aplicada:

Implementamos un motor de saneamiento en el `BggSyncService` y escalamos los tipos de datos en la base de datos dinámicamente (`TEXT`, `VARCHAR`). Protegimos el proceso con Pruebas de Integración y `@Rollback`.



[INSERTA AQUÍ LA CAPTURA DEL IMPORTADOR MÁGICO]
(La pantalla donde buscas por nombre y se rellena todo)

7. Dificultades y Soluciones (II) – Frontend



[INSERTA AQUÍ LA CAPTURA DE LA GESTIÓN DE SOCIOS]
(Para mostrar una interfaz compleja cargada de datos)

*Reto: Congelación de la Interfaz (UI Freeze)

Al realizar peticiones de red pesadas, la interfaz de JavaFX se congelaba y el usuario no podía interactuar.

Solución Aplicada:

Implementación estricta de multithreading. Usamos `Task` para aislar las llamadas HTTP en hilos secundarios, y devolvimos los resultados de forma segura a la pantalla usando `Platform.runLater()`.

6. DEMOSTRACIÓN EN VIVO

Flujo Completo de Ejecución

- ▶ 1. Importación automatizada desde BoardGameGeek.
- ▶ 2. Creación de un Ejemplar físico en el inventario.
- ▶ 3. Filtrado reactivo Zero-Lag en el catálogo principal.

8. Mejoras Futuras



App Móvil para Socios

Al tener una API REST completamente desacoplada, el siguiente paso lógico es desarrollar un cliente en Kotlin o Flutter para que los socios consulten el catálogo desde su bolsillo.



Sistema Kiosko (Código de Barras)

Aprovechar el campo `codigo_local` de los ejemplares para integrar un lector óptico, permitiendo automatizar el alta y devolución de préstamos mediante escaneo.

9. Conclusiones Personales

Este Trabajo de Fin de Grado me ha permitido enfrentarme al ciclo de vida completo de un software profesional:

- ✓ Comprensión profunda de la **arquitectura y patrones de diseño** (MVC, REST, DTOs).
- ✓ Manejo real de **concurrency y bases de datos** transaccionales.
- ✓ Valor de escribir **código limpio, escalable** y protegido contra fallos externos (APIs).



Objetivo Cumplido

LudiGest no es solo un proyecto académico; es una solución real y funcional lista para ser desplegada en producción.

Muchas Gracias

Turno abierto de preguntas

 Alejandro Muñoz de la Sierra